

Artificial Intelligence Lab 02

AL-2002 | Instructor: Saad Ahmad | Semester: Spring 2026

Student: Mehr Ali **Topic:** AI Agents — Simple Reflex Agents

Task 1: Fire Alarm System — Simple Reflex Agent

- smoke detected → Sound Alarm + Activate Sprinklers
- high temperature → Sound Alarm + Call Fire Department

```
In [ ]: # =====
# Task 1: Fire Alarm System - Simple Reflex Agent
# =====

percepts = ['smoke_detected', 'high_temperature', 'normal']
states    = ['smoke_present', 'high_temp_present', 'safe']
rules     = {
    'smoke_present'      : 'Sound Alarm + Activate Sprinklers',
    'high_temp_present' : 'Sound Alarm + Call Fire Department',
    'safe'               : 'No Action - System is Normal'
}

def interpret_input(percept_value):
    if percept_value == 'smoke_detected':
        return 'smoke_present'
    elif percept_value == 'high_temperature':
        return 'high_temp_present'
    else:
        return 'safe'

def rule_match(state):
    return rules.get(state, 'Unknown State - No Rule Found')

def fire_alarm_agent(percept):
    state = interpret_input(percept)
    action = rule_match(state)
    return state, action

print("=" * 55)
print("  FIRE ALARM SYSTEM - Simple Reflex Agent")
print("=" * 55)
print("Available sensor inputs:")
print("  1. smoke_detected")
```

```

print(" 2. high_temperature")
print(" 3. normal")
print("-" * 55)

while True:
    sensor_input = input("\nEnter sensor reading (or 'exit' to quit): ").strip().lower()

    if sensor_input == 'exit':
        print("\nShutting down Fire Alarm System. Stay safe!")
        break

    if sensor_input not in percepts:
        print(" [!] Invalid input. Please enter: smoke_detected, high_temperature, normal")
        continue

    current_state, action_taken = fire_alarm_agent(sensor_input)

    print(f"\n Sensor Reading : {sensor_input}")
    print(f" Current State : {current_state}")
    print(f" Agent Action : {action_taken}")
    print("-" * 55)

```

```

=====
FIRE ALARM SYSTEM – Simple Reflex Agent
=====
Available sensor inputs:
1. smoke_detected
2. high_temperature
3. normal
-----

```

Task 2: Automatic Watering System — Simple Reflex Agent

```

In [ ]: # =====
# Task 2: Automatic Watering System - Simple Reflex Agent
# =====

import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
logger = logging.getLogger('WateringSystem')

class MoistureSensor:
    def __init__(self, dry_threshold=30, wet_threshold=70):
        if not (0 < dry_threshold < wet_threshold < 100):
            raise ValueError(
                f"Invalid thresholds! Must satisfy: 0 < dry({dry_threshold}) < wet({wet_threshold}) < 100"
            )
        self.dry_threshold = dry_threshold
        self.wet_threshold = wet_threshold
        logger.info(f"Sensor ready | Dry: <{dry_threshold}% | Moist: {wet_threshold}%")

```

```

def read(self, moisture_value):
    if not isinstance(moisture_value, (int, float)):
        raise TypeError(f"Expected a number, got: {type(moisture_value).__name__}")
    if not (0 <= moisture_value <= 100):
        raise ValueError(f"Value {moisture_value}% is out of range. Must be 0 to 100")

    if moisture_value < self.dry_threshold:
        state = 'dry'
    elif moisture_value <= self.wet_threshold:
        state = 'moist'
    else:
        state = 'wet'

    logger.info(f"Reading: {moisture_value}% -> {state.upper()} SOIL")
    return state

class WateringController:
    def __init__(self):
        self.is_active = False
        logger.info("Watering Controller ready. System is OFF.")

    def activate(self):
        if not self.is_active:
            self.is_active = True
            logger.info("System ACTIVATED – Watering plants now.")
        else:
            logger.info("System already ON.")

    def deactivate(self):
        if self.is_active:
            self.is_active = False
            logger.info("System DEACTIVATED – Watering stopped.")
        else:
            logger.info("System already OFF.")

    def get_status(self):
        return "ON" if self.is_active else "OFF"

class AutomaticWateringAgent:
    def __init__(self, dry_threshold=30, wet_threshold=70):
        self.sensor = MoistureSensor(dry_threshold, wet_threshold)
        self.controller = WateringController()
        logger.info("Automatic Watering Agent is ready.\n")

    def perceive(self, moisture_value):
        return self.sensor.read(moisture_value)

    def decide_and_act(self, state):
        print()
        if state == 'dry':
            print(" [RULE] Dry soil -> Activating watering system...")
            self.controller.activate()
        elif state == 'moist':

```

```

        print(" [RULE] Moist soil -> Keeping watering system OFF...")
        self.controller.deactivate()
    elif state == 'wet':
        print(" [RULE] Wet soil -> Deactivating system to prevent waterlogging")
        self.controller.deactivate()
    else:
        logger.warning(f"Unknown state: {state}")
        print(" [!] Unknown state - No action taken.")

def run(self, moisture_value):
    print("-" * 60)
    try:
        state = self.perceive(moisture_value)
        self.decide_and_act(state)
        print(f"\n Watering System Status: [ {self.controller.get_status()} ]")
    except (ValueError, TypeError) as error:
        logger.error(f"Input error: {error}")
        print(f" [ERROR] {error}")
    print("-" * 60)

print("=" * 60)
print("  AUTOMATIC WATERING SYSTEM - Simple Reflex Agent")
print("=" * 60)
print("Default Thresholds: Dry < 30% | Moist 30-70% | Wet > 70%")
print("=" * 60)
print()

agent = AutomaticWateringAgent(dry_threshold=30, wet_threshold=70)

print("Agent running. Type a moisture value or 'exit' to quit.\n")

while True:
    user_input = input("Enter moisture level (0-100) or 'exit': ").strip()

    if user_input.lower() == 'exit':
        print("\nShutting down. Goodbye!")
        break

    try:
        moisture_level = float(user_input)
        agent.run(moisture_level)
    except ValueError:
        print(" [!] Please enter a valid number between 0 and 100.")

```